

Week .2

Course Name : Problem Solving and Testing

Course Code : 10211CS227

Name : K.Siddartha

Vtu No : VTU28557

1. Build Array from Permutation

2. Shuffle the Array

3. Find the Highest Altitude

4. Java Dequeue

5. Java Hashset

1. Build Array from Permutation

PROGRAM:

```
class Solution {  
    public int[] buildArray(int[] nums) {  
        int[] ans = new int[nums.length];  
  
        for (int i = 0; i < nums.length; i++) {  
            ans[i] = nums[nums[i]];  
        }  
  
        return ans;  
    }  
}
```

OUTPUT:

☒ Testcase | ☒ Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2

Input

nums =
[0,2,1,5,3,4]

Output

[0,1,2,4,5,3]

Expected

[0,1,2,4,5,3]

2. Shuffle the Array

PROGRAM:

```
class Solution {  
    public int[] shuffle(int[] nums, int n) {  
        int[] ans = new int[2 * n];  
  
        for (int i = 0; i < n; i++) {  
            ans[2 * i] = nums[i];  
            ans[2 * i + 1] = nums[i + n];  
        }  
  
        return ans;  
    }  
}
```

OUTPUT:

☒ Testcase ☒ Test Result

Accepted Runtime: 0 ms

☒ Case 1 ☒ Case 2 ☒ Case 3

Input

nums =
[2, 5, 1, 3, 4, 7]

n =
3

Output

[2, 3, 5, 4, 1, 7]

Expected

[2, 3, 5, 4, 1, 7]

☒ Testcase | [> Test Result](#)

Accepted

Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

nums =
[1, 2, 3, 4, 4, 3, 2, 1]

n =
4

Output

[1, 4, 2, 3, 3, 2, 4, 1]

Expected

[1, 4, 2, 3, 3, 2, 4, 1]

☒ Testcase | [> Test Result](#)

Accepted

Runtime: 0 ms

☒ Case 1

☒ Case 2

☒ Case 3

Input

nums =
[1, 1, 2, 2]

n =
2

Output

[1, 2, 1, 2]

Expected

[1, 2, 1, 2]

3. Find the Highest Altitude

PROGRAM:

```
class Solution {  
    public int largestAltitude(int[] gain) {  
        int altitude = 0;  
        int highest = 0;  
  
        for (int i = 0; i < gain.length; i++) {  
            altitude += gain[i];  
  
            if (altitude > highest) {  
                highest = altitude;  
            }  
        }  
  
        return highest;  
    }  
}
```

OUTPUT:

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

gain =
[-5, 1, 5, 0, -7]

Output

1

Expected

1

✓ testcase | ✓ test result

Accepted Runtime: 0 ms

✓ Case 1

✓ Case 2

Input

gain =
[-4, -3, -2, -1, 4, 3, 2]

Output

0

Expected

0

4. Java Dequeue

PROGRAM:

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();
        int m = sc.nextInt();

        int[] arr = new int[n];

        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        Deque<Integer> deque = new ArrayDeque<>();
        Map<Integer, Integer> freq = new HashMap<>();

        int maxUnique = 0;

        for (int i = 0; i < n; i++) {
            deque.addLast(arr[i]);
```

```
freq.put(arr[i], freq.getDefault(arr[i], 0) + 1);

// Keep window size equal to m
if (deque.size() > m) {
    int removed = deque.removeFirst();

    freq.put(removed, freq.get(removed) - 1);

    if (freq.get(removed) == 0) {
        freq.remove(removed);
    }
}

// Number of unique elements in current window
if (deque.size() == m) {
    maxUnique = Math.max(maxUnique, freq.size());
}
}

System.out.println(maxUnique);
}
}
```


OUTPUT:

	 Console	 I/O	 AI Agent	
	6 3			
	5 3 2 6 1 8			
	3			

5. Java Hashset

PROGRAM:

```
import java.util.*;

public class Main {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        int n = sc.nextInt();

        Set<String> set = new HashSet<>();

        for (int i = 0; i < n; i++) {

            String first = sc.next();

            String second = sc.next();

            String pair = first + " " + second;

            set.add(pair);

            System.out.println(set.size());

        }

        sc.close();

    }

}
```

OUTPUT:

	 Console	 I/O	 AI Agent	
	4			
	SURYA			
	VAMSHI			
	1			
	SIDDHU			
	SIVA			
	2			